

Actividad Autónoma - Sesión 8

Pruebas de dependencias entre Categorías y Productos

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	8 · 1 de octubre de 2026
Modalidad	Individual, sin acompañamiento docente	Plazo	7 de octubre de 2026, 23:59
Entrega	Aula virtual + repositorio Git	Puntaje	20 puntos (evaluación de sesión)
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2
Estudiante	Kevin Eduardo Del Carpio Alegría	Proyecto	PharmaSoft

B1. Resumen de la ejecución

Se evaluaron 18 casos: 14 de la guía y cuatro propios (A-07, C-05, C-06 y B-05). Se contrastaron SPA y API. Las verificaciones y regresiones complementarias de Codex se identifican en sus registros.

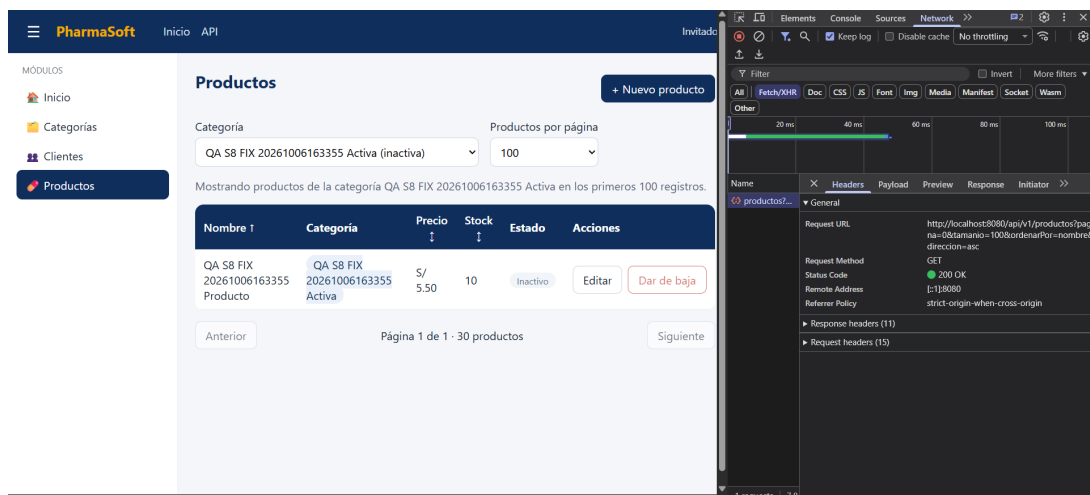
Grupo	Casos API	Pasan	Fallan
Altas	7	5	2
Cambios	6	2	4
Bajas	5	5	0
Total	18	12	6

La tabla corresponde a la línea base. Los seis fallos originaron seis fichas y cinco causas: A-04 y C-04 comparten la validación ausente del estado de la categoría. La regresión posterior se registra por separado.

Criterio: impedir la desactivación con productos activos y la eliminación con cualquier producto asociado. La baja lógica conserva la clasificación histórica.

Regresión: con apoyo de Codex se incorporaron validaciones en "feature/productos-delcarpio". Los registros posteriores muestran 19 pruebas automatizadas y 28 verificaciones API satisfactorias, separadas de la línea base.

Evidencia: capturas y registros por caso; el encuadre limita la lectura de B-04. No se verificó concurrencia ni se incorporó unicidad en la base de datos.



The screenshot shows the PharmaSoft application interface. On the left, there's a sidebar with 'Inicio', 'Categorías', 'Clientes', and 'Productos'. The main area displays the 'Productos' page with a filter for 'QA S8 FIX 20261006163355 Activa (inactiva)' and a 'Productos por página' dropdown set to 100. Below this, a table lists products, including 'QA S8 FIX 20261006163355 Activa'. To the right, a browser's developer console shows a network request to 'http://localhost:8080/api/v1/productos/pagina=0&tamaño=100&ordenarPor=nombre&direccion=asc' with a status of 200 OK.

Parte C: acceso a productos de la categoría 69 y tamaño 100.

B2. Fichas de hallazgos: altas

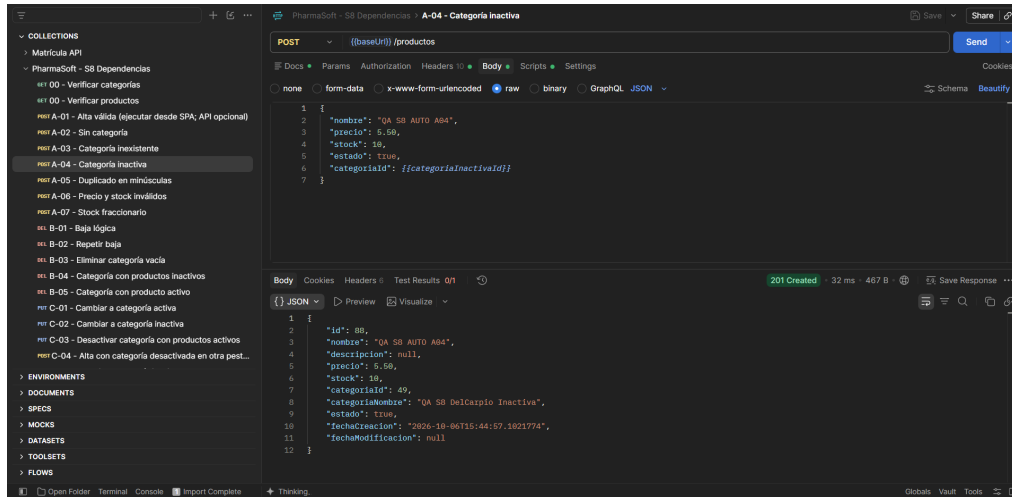
H-01. Categoría inactiva aceptada

Caso: A-04. **Severidad:** alta, por inconsistencia de datos. **Descripción:** POST devolvió 201 para categoría 49 inactiva (producto 88), aunque la SPA la oculta. El registro activo contradice la regla de categoría activa.

Causa probable y capa: Backend, ProductoServiceImpl, create(): comprobaba la existencia de la categoría, pero no su estado. Las clases citadas pertenecen al backend.

Corrección propuesta e incorporada: Validar estado en create/update antes de mapear o guardar; lanzar ReglaNegocioException (409). Compartir bloqueo de categoría con su desactivación para coordinar escrituras.

Evidencia: captura del caso y registros de verificación.



A-04: Postman aceptó la categoría inactiva con 201.

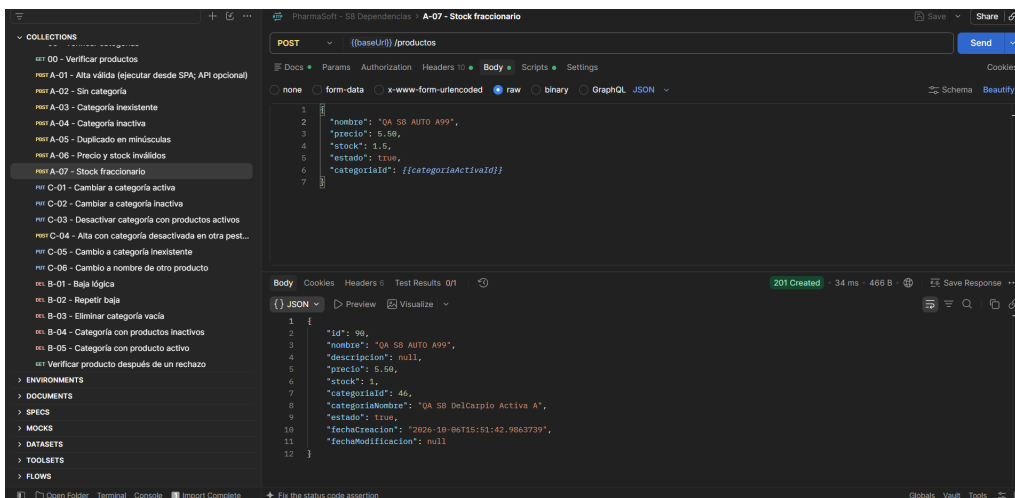
H-02. Stock fraccionario truncado

Caso: A-07 (propio). **Severidad:** alta, por inconsistencia de datos. **Descripción:** La SPA bloqueó 1.5; la API creó producto 90 (QA S8 AUTO A99) con stock 1. El nombre cambió tras una colisión; no se atribuye ese rechazo al stock.

Causa probable y capa: Backend, ProductoRequestDTO, campo stock (Integer): Jackson convertía el decimal antes de @Min. La validación comprobaba el entero ya convertido; JsonValidationConfig controla ahora esa conversión.

Corrección propuesta e incorporada: Deshabilitar ACCEPT_FLOAT_AS_INT con Jackson 3 y devolver 400 desde GlobalExceptionHandler.handleInvalidJson. Probar POST/PUT con fracciones y enteros válidos.

Evidencia: captura del caso y registros de verificación.



A-07: stock 1.5 enviado y stock 1 devuelto.

B2. Fichas de hallazgos: cambios de categoría

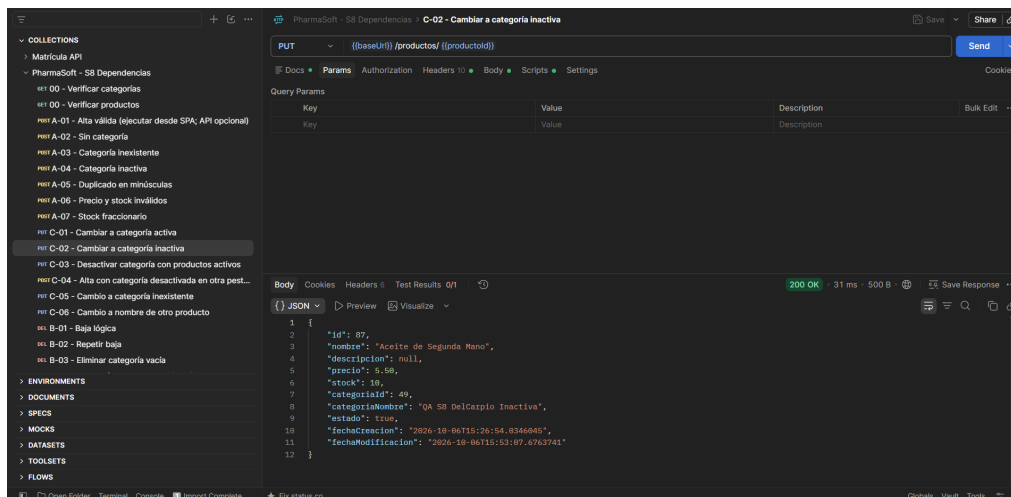
H-03. Reasignación a categoría inactiva

Caso: C-02. **Severidad:** alta, por inconsistencia de datos. **Descripción:** PUT del producto 87 devolvió 200 al moverlo de categoría 47 a 49, conservándolo activo. La prueba de la SPA usó el producto de prueba 73 y bloqueó el guardado.

Causa probable y capa: Backend, ProductoServiceImpl, update(): faltaba validar el estado de la categoría destino.

Corrección propuesta e incorporada: Rechazar con 409 antes de modificar la entidad y comprobar por GET que se conservan los datos anteriores.

Evidencia: captura del caso y registros de verificación.



C-02: PUT aceptó el cambio a categoría inactiva.

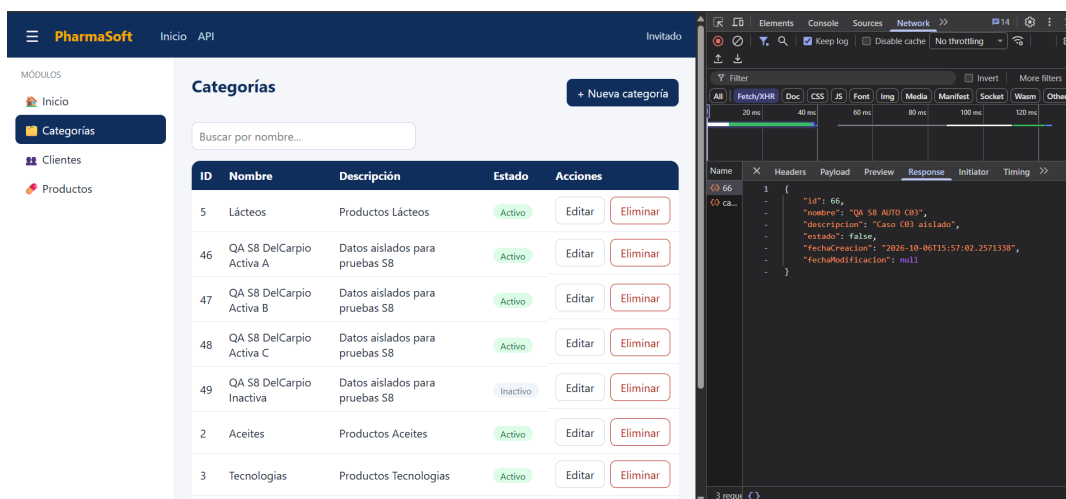
H-04. Desactivación con productos activos

Caso: C-03. **Severidad:** alta, por inconsistencia de datos. **Descripción:** PUT de categoría 66 devolvió 201 y estado false; GET confirmó que producto 91 seguía activo. La edición además usaba código de creación.

Causa probable y capa: Backend, CategoriaServiceImpl, update(), no consultaba productos activos; CategoriaController, update(), devolvía 201.

Corrección propuesta e incorporada: Consultar existsByCategorialdAndEstadoTrue, rechazar con 409 y preservar ambos estados. Bloquear la misma categoría dentro de la transacción; devolver 200 para una edición válida.

Evidencia: captura del caso y registros de verificación.



C-03: categoría guardada con estado false.

B2. Fichas de hallazgos: nombre y formulario obsoleto

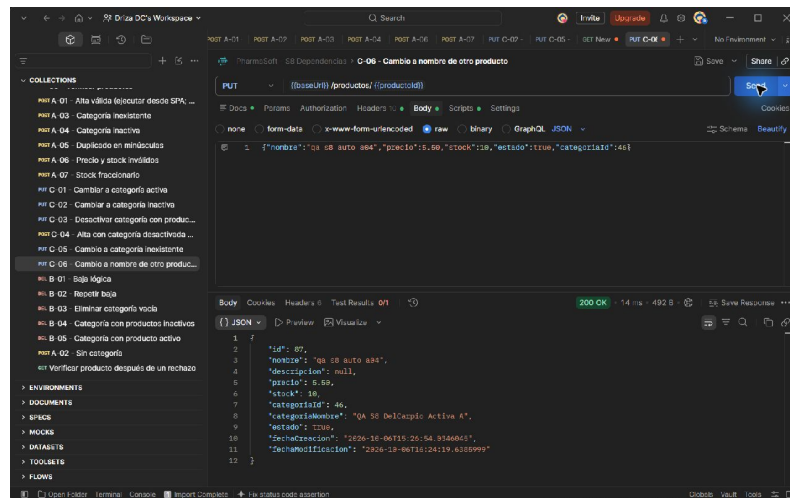
H-05. Nombre duplicado al actualizar

Caso: C-06 (propio). **Severidad:** alta, por inconsistencia de datos. **Descripción:** SPA y API aceptaron qa s8 auto a04 para producto 87, duplicando el nombre de producto 88 al comparar sin distinguir mayúsculas y minúsculas.

Causa probable y capa: Backend, ProductoServiceImpl, update(), no consultaba existsByNombreIgnoreCaseAndIdNot antes de modificar la entidad.

Corrección propuesta e incorporada: Eliminar espacios al inicio y al final y comparar sin distinguir mayúsculas y minúsculas, excluyendo el ID propio, y rechazar con 409 antes de guardar. La unicidad simultánea requiere además una restricción de base de datos y saneamiento de duplicados históricos; eso no se implementó.

Evidencia: captura del caso y registros de verificación.



C-06: actualización duplicada aceptada con 200.

H-06. Formulario obsoleto crea vínculo inválido

Caso: C-04. **Severidad:** alta, por inconsistencia de datos. **Descripción:** La SPA cargó categoría 67 activa. Otra pestaña la desactivó y el formulario original creó producto 92; un POST complementario creó producto 93 (201).

Causa probable y capa: Backend, ProductoServiceImpl, create(), validaba existencia, pero no estado al guardar. Comparte causa con H-01: el formulario de productos usa una lista local cargada antes del cambio.

Corrección propuesta e incorporada: Revalidar estado en la transacción y coordinar con CategoriaServiceImpl.update() mediante el bloqueo de categoría. Responder 409 sin guardar y mantener el formulario con mensaje claro.

Evidencia: captura del caso y registros de verificación.

C-04: formulario utilizado en la secuencia de dos pestañas; los JSON vinculados confirman el producto creado.

B3. Cobertura de reglas

Se distingue la validación que protege los datos en la API de la prevención local en la SPA. Mostrar un error recibido del servidor no constituye una segunda validación independiente.

Regla	Línea base	Tras corrección
Existencia de categoría	Ambas: lista/validación local y API	Ambas; API comprueba existencia
Categoría activa	Solo SPA	Ambas; API revalida al guardar
Nombre único	Solo API en alta; ninguna en cambio	Solo API en alta y cambio
No eliminar categoría con productos	Solo API	Solo API
Baja lógica / repetir baja	Ambas: API cambia estado; SPA impide repetir	Ambas; API conserva rechazo 409
No desactivar con activos	Ninguna	Solo API

En el formulario de productos, `guardar()` comprueba la categoría de la lista local. En `ProductoServiceImpl`, `create/update` validan la referencia y el nombre. `CategoriaServiceImpl` protege la desactivación y eliminación mediante consultas de `ProductoRepository`. En el listado de productos, `darDeBaja()` impide repetir la acción sobre una fila inactiva; la API mantiene esa regla aunque la petición llegue desde Postman.

B4. Filtro por categoría

En el listado, `ProductoList` filtra `resultado().contenido` después de recibir una página del servidor. Por tanto, oculta registros de esa página, pero no consulta productos de otras páginas; el total de la paginación sigue siendo global. Cargar 100 reduce el problema, pero no convierte el filtro en global.

Propuesta completa: se propone agregar `categoriald` opcional a `GET /api/v1/productos`, conservando `pagina`, `tamano`, `ordenarPor` y `direccion`. En `ProductoRepository` añadir `Page<Producto> findByCategoriald(Long categoriald, Pageable pageable)`. El servicio backend elige `findAll(pageable)` cuando no hay categoría y el método filtrado cuando sí la hay; el total debe corresponder a la consulta filtrada. `ProductoService.listar()` del frontend añade `categoriald` como parámetro HTTP opcional. `ProductoList` reinicia `pagina` a 0 al cambiar filtro y vuelve a consultar; elimina el filtrado local y presenta el total filtrado. Esta mejora global es una propuesta pendiente de implementación.

Mejora de la Parte C: se incorporó, con apoyo de Codex, el enlace Ver productos que usa `routerLink` y `queryParams`. `ProductoList` recibe `categoriald` mediante `input` y `withComponentInputBinding`, selecciona el filtro y solicita tamaño 100. La nota explica los primeros 100 registros y cambia al avanzar de página. La navegación y la recepción del parámetro se configuran en los componentes de categorías y productos y en la configuración de rutas.

B5. Análisis

1. ¿Por qué no basta con ocultar categorías inactivas? A-04 y C-02 demostraron que Postman puede enviar referencias excluidas del formulario. `ProductoServiceImpl.create/update` deben validar el estado al guardar, independientemente de la selección local.

2. ¿Qué debe ocurrir al desactivar una categoría con productos activos? Debe impedirse para conservar una clasificación válida de los artículos disponibles para venta. `CategoriaServiceImpl.update()` consulta `ProductoRepository.existsByCategorialdAndEstadoTrue` y exige reasignación o baja explícita.

3. ¿Los productos dados de baja deben impedir eliminar su categoría? Sí: la baja lógica conserva la clasificación para reportes e historial de ventas. `CategoriaServiceImpl.delete()` y `ProductoRepository.existsByCategoriald` deben preservar cualquier referencia, incluido el producto inactivo.

4. ¿Cuándo deben validarse las dependencias? C-04 demuestra que el estado puede cambiar después de abrir el formulario. `ProductoServiceImpl.create()` debe revalidarlo en la transacción y coordinar su bloqueo con `CategoriaServiceImpl.update()`. La selección local solo orienta al usuario.

5. ¿Qué cambia en la SPA al corregir A-04? Se mantiene el filtro de categorías activas. `mensajeError` y el `signal error` muestran el nuevo 409 sin perder los datos. Deben actualizarse los resultados esperados y verificarse el rechazo del formulario obsoleto.